14 软件和硬件性能的当前和未来趋势

软件性能

- 人工智能和机器学习的兴起将继续推动软件性能的进步。
- 软件开发人员将需要更多地了解并利用硬件加速器来提高性能。
- 软件架构将变得更加复杂，以充分利用多核处理器和异构计算平台。

硬件性能

- 摩尔定律的放缓将迫使硬件制造商寻找新的方法来提高性能。
- 3D 堆叠和 chiplet 设计等新技术将使芯片能够在更小的空间内容纳更多的晶体管。
- 光子和量子计算等新兴技术可能会在未来几年内带来重大突破。

软件和硬件性能的协同发展

- 软件和硬件开发人员将需要更加紧密地合作，以充分利用未来的计算平台。
- 编译器和操作系统等软件工具将需要不断改进，以更好地利用硬件功能。
- 性能分析和调优将变得更加重要，以确保软件能够充分发挥硬件的潜力。

以下是一些具体的趋势示例：

- **人工智能和机器学习**

人工智能和机器学习 (AI/ML) 应用程序对性能有着极高的要求。为了满足这些要求，软件开发人员将需要更多地了解并利用硬件加速器。例如，GPU 和 TPU 能够比传统的 CPU 或 CPU 更快地执行 AI/ML 任务。

- **多核处理器和异构计算平台**

多核处理器和异构计算平台正在变得越来越普遍。为了充分利用这些平台，软件架构将变得更加复杂。例如，软件开发人员将需要使用线程和并行编程技术来充分利用多个内核和加速器。

- **3D 堆叠和 chiplet 设计**

3D 堆叠和 chiplet 设计等新技术使芯片能够在更小的空间内容纳更多的晶体管。这将导致更高的性能和更低的功耗。

- **光子和量子计算**

光子和量子计算等新兴技术可能会在未来几年内带来重大突破。这些技术有可能提供比传统计算平台高得多的性能。

结论

软件和硬件性能的趋势将继续快速发展。软件开发人员和硬件制造商将需要共同努力，以充分利用未来的计算平台。

后记

感谢您阅读完整本书。希望您喜欢并从中有所收获。如果本书能帮助您解决实际问题，我会更加高兴。在这种情况下，我将把它视为成功，并证明我的努力没有白费。在您继续努力之前，让我简要强调本书的要点并为您提供最终建议：

- 硬件性能不像几十年前那样快速增长了。性能调优变得比过去 40 年来更加重要。它将是未来获得性能提升的主要驱动力之一。
- 软件默认情况下没有最佳性能。存在某些限制阻止应用程序发挥其全部性能潜力。硬件和软件组件都存在这样的限制。

- Donald Knuth 有句名言：“过早优化是所有问题的根源”。但反过来也经常是真的。推迟性能工程工作可能为时已晚，并可能像过早优化一样造成同样的危害。在设计未来产品时，不要忽视性能方面。
- 现代 CPU 的性能不是确定性的，取决于许多因素。有意义的性能分析应该考虑噪音并使用统计方法分析性能测量。
- 了解 CPU 微架构可能有助于理解您进行实验的结果。但是，在对代码进行特定更改时，不要过分依赖这些知识。您的心智模型永远不可能像 CPU 内部实际设计那样准确。预测特定代码片段的性能几乎是不可能的。
- 一定要测量！
- 性能调优很难，因为它没有预定的步骤需要遵循，没有算法。工程师需要从不同的角度解决问题。了解可用的性能分析方法和工具（硬件和软件）。我强烈建议拥抱 Roofline 模型和 TMA 方法，如果它们在您的平台上可用。它将帮助您将您的工作引导到正确的方向。此外，要知道何时可以利用其他硬件性能监控功能，例如 LBR、PEBS 和 PT。
- 了解应用程序性能的限制因素以及可能的修复方法。第二部分涵盖了一些针对每种类型 CPU 性能瓶颈的基本优化：前端受限、后端受限、回退、错误推测。使用第 8-11 章查看当您的应用程序属于上述四种类别之一时可用的选项。
- 如果修改带来的好处可以忽略不计，您应该保持代码最简单和最干净的形式。
- 有时，在系统上提高性能的修改会在另一个系统上减慢执行速度。请确保在您关心的所有平台上测试您的更改。

我希望这本书能帮助您更好地理解应用程序的性能和整体 CPU 性能。当然，它无法涵盖您在进行性能优化时可能遇到的所有情况。我的目标是为您提供一个起点，并向您展示在现代 CPU 上进行性能分析和调整的潜在选项和策略。

如果您喜欢阅读本书，请务必将其传递给您的朋友和同事。如果您能在社交媒体平台上宣传本书，我将不胜感激。

我非常欢迎您通过电子邮件 dendibakh@gmail.com 提供反馈。请告诉我您对本书的想法、评论和建议。我将在我的博客 https://easyperf.net/contact/ 上发布所有更新和未来有关本书的信息。

祝您性能调优愉快！

术语表

AOS Array Of Structures

BB Basic Block

BIOS Basic Input Output System

CI/CD Contiguous Integration/ Contiguous Development

CPI Clocks Per Instruction

CPU Central Processing Unit

DSB Decoded Stream Buffer

DRAM Dynamic Random-Access Memory

DTLB Data Translation Lookaside Buffer

EBS Event-Based Sampling

FLOPS FLoating-point Operations Per Second

FPGA Field-Programmable Gate Array

GPU Graphics processing unit

HFT High-Frequency Trading

HPC High Performance Computing

HW Hardware

I/O Input/Output

IDE Integrated Development Environment

ILP Instruction-Level Parallelism

IPC Instructions Per Clock cycle

IPO Inter-Procedural Optimizations

ITLB Instruction Translation Lookaside Buffer

LBR Last Branch Record

LLC Last Level Cache

LSD Loop Stream Detector

MSR Model Specific Register

MS-ROM Microcode Sequencer Read-Only Memory

NUMA Non-Uniform Memory Access

OS Operating System

PEBS Processor Event-Based Sampling

PGO Profile Guided Optimizations

PMC Performance Monitoring Counter

PMI Performance Monitoring Interrupt

PMU Performance Monitoring Unit

PT Processor Traces

RAT Register Alias Table

ROB ReOrder Buffer

SIMD Single Instruction Multiple Data

SMT Simultaneous MultiThreading

SOA Structure Of Arrays

SW Software

TLB Translation Lookaside Buffer

TMA Top-down Microarchitecture Analysis

TSC Time Stamp Counter

μop MicroOperation

主要 CPU 微架构列表

在下面的表格中，我们展示了来自 Intel、AMD 和基于 ARM 的供应商的最新 ISA 和微架构。当然，并不是所有的设计都列在这里。我们只包括了那些在本书中引用的或者代表了平台演变中的重大转变的设计。

Table 12: 最近的英特尔 Core 微架构列表。

名称	三字母缩写	发布年份	支持的 ISA 客户端/服务器芯片
Nehalem	NHM	2008	SSE4.2
Sandy Bridge	SNB	2011	AVX
Haswell	HSW	2013	AVX2
Skylake	SKL	2015	AVX2 / AVX512
Sunny Cove	SNC	2019	AVX512
Golden Cove	GLC	2021	AVX2 / AVX512
Redwood Cove	RWC	2023	AVX2 / AVX512

Table 13: 最近的 AMD 微架构列表。

名称	发布年份	支持的 ISA
Streamroller	2014	AVX
Excavator	2015	AVX2
Zen	2017	AVX2
Zen2	2019	AVX2
Zen3	2020	AVX2
Zen4	2022	AVX512

Table 14: 最近的 ARM ISA 列表，以及它们自己和第三方的实现。

ISA	ISA 发布年份	ARM 微架构 (最新)	第三方微架构
ARMv8-A	2011	Cortex-A73	Apple A7-A10; Quallcomm Kryo; Samsung M1/M2/M3
ARMv8.2-A	2016	Neoverse N1; Cortex-X1	Apple A11; Samsung M4; Ampere Altra
ARMv8.4-A	2017	Neoverse V1	AWS Graviton3; Apple A13, M1
ARMv9.0-A (64 位)	2018	Neoverse N2; Neoverse V2; Cortex X3	Microsoft Cobalt 100; NVIDIA Grace
ARMv8.6-A (64 位)	2019	—	Apple A15, A16, M2, M3
ARMv9.2-A	2020	Cortex X4	—

References

[Advanced Micro Devices, 2021] Advanced Micro Devices (2021). Processor Programming Reference (PPR) for AMD family 19h model 01h (55898). B1 Rev 0.50. https://www.amd.com/content/dam/amd/en/documents/epyc-technical-docs/programmer-references/55898_B1_pub_0_50.zip

[Advanced Micro Devices, 2022] Advanced Micro Devices (2022). AMD64 technology platform quality of service extensions. Pub. 56375, rev 1.01. https://www.amd.com/content/dam/amd/en/documents/processor-tech-docs/other/56375_1_03_PUB.pdf

[Akinshin, 2019] Akinshin, A. (2019). Pro .NET Benchmarking (1 ed.). Apress. https://doi.org/10.1007/978-1-4842-4941-3

[Alam et al., 2019] Alam, M., Gottschlich, J., Tatbul, N., Turek, J. S., Mattson, T., & Muzahid, A. (2019). A zero-positive learning approach for diagnosing software performance regressions. Advances in Neural Information Processing Systems 32, 11627 – 11639. Curran Associates, Inc. http://papers.nips.cc/paper/9337-a-zero-positive-learning-approach-for-diagnosing-software-performance-regressions.pdf

[AMD, 2023] AMD (2023). AMD64 Architecture Programmer's Manual. Advanced Micro Devices, Inc. https://www.amd.com/content/dam/amd/en/documents/processor-tech-docs/programmer-references/24593.pdf

[AMD, 2024] AMD (2024). AMD uProf User Guide, Revision 4.2. Advanced Micro Devices, Inc. https://www.amd.com/content/dam/amd/en/documents/developer/version-4-2-documents/uprof/uprof-user-guide-v4.2.pdf

[Arm, 2022a] Arm (2022a). Arm Architecture Reference Manual Supplement Armv9. Arm Limited. https://documentation-service.arm.com/static/632dbdace68c6809a6b41710?token=

[Arm, 2022b] Arm (2022b). Arm Neoverse™ V1 PMU Guide, Revision: r1p2. Arm Limited. https://developer.arm.com/documentation/PJDOC-1063724031-605393/2-0/?lang=en

[Arm, 2023a] Arm (2023a). Arm Neoverse V1 Core: Performance Analysis Methodology. Arm Limited. https://armkeil.blob.core.windows.net/developer/Files/pdf/white-paper/neoverse-v1-core-performance-analysis.pdf

[Arm, 2023b] Arm (2023b). Arm Statistical Profiling Extension: Performance Analysis Methodology. Arm Limited. https://developer.arm.com/documentation/109429/latest/

[Chen et al., 2016] Chen, D., Li, D. X., & Moseley, T. (2016). Autofdo: Automatic feedback-directed optimization for warehouse-scale applications. CGO 2016 Proceedings of the 2016 International Symposium on Code Generation and Optimization, 12 – 23. https://ieeexplore.ieee.org/document/7559528

[Cooper & Torczon, 2012] Cooper, K. & Torczon, L. (2012). Engineering a Compiler. Morgan Kaufmann. Morgan Kaufmann. https://books.google.co.in/books?id=CGTOlAEACAAJ

[Curtsinger & Berger, 2013] Curtsinger, C. & Berger, E. D. (2013). Stabilizer: Statistically sound performance evaluation. Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '13, 219–228. https://doi.org/10.1145/2451116.2451141

[Curtsinger & Berger, 2018] Curtsinger, C. & Berger, E. D. (2018). Coz: Finding code that counts with causal profiling. Commun. ACM, 61(6), 91–99. https://doi.org/10.1145/3205911

[Daly et al., 2020] Daly, D., Brown, W., Ingo, H., O'Leary, J., & Bradford, D. (2020). The use of change point detection to identify software performance regressions in a continuous integration system. Proceedings of the ACM/SPEC International Conference on Performance Engineering, ICPE '20, 67–75. https://doi.org/10.1145/3358960.3375791

[domo.com, 2017] domo.com (2017). Data Never Sleeps 5.0. Domo, Inc. https://www.domo.com/learn/data-never-sleeps-5?aid=ogsm072517_1&sf100871281=1

[Du et al., 2010] Du, J., Sehrawat, N., & Zwaenepoel, W. (2010). Performance profiling in a virtualized environment. Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing, HotCloud'10, 2. https://www.usenix.org/legacy/event/hotcloud10/tech/full_papers/Du.pdf

[Fog, 2004] Fog, A. (2004). Optimizing software in c++: An optimization guide for windows, linux and mac platforms. https://www.agner.org/optimize/optimizing_cpp.pdf

[Fog, 2012] Fog, A. (2012). The microarchitecture of intel, amd and via cpus: An optimization guide for assembly programmers and compiler makers. Copenhagen University College of Engineering. https://www.agner.org/optimize/microarchitecture.pdf

[Gregg, 2013] Gregg, B. (2013). Systems Performance: Enterprise and the Cloud (1st ed.). Prentice Hall Press.

[Grosser et al., 2012] Grosser, T., Größlinger, A., & Lengauer, C. (2012). Polly - performing polyhedral optimizations on a low-level intermediate representation. Parallel Process. Lett., 22.

[Hennessy, 2018] Hennessy, J. L. (2018). The future of computing. https://youtu.be/Azt8Nc-mtKM?t=329

[Hennessy & Patterson, 2017] Hennessy, J. L. & Patterson, D. A. (2017). Computer Architecture, Sixth Edition: A Quantitative Approach (6th ed.). Morgan Kaufmann Publishers Inc.

[Ingo & Daly, 2020] Ingo, H. & Daly, D. (2020). Automated system performance testing at mongodb. Proceedings of the Workshop on Testing Database Systems, DBTest '20. https://doi.org/10.1145/3395032.3395323

[Intel, 2023a] Intel (2023a). CPU Metrics Reference. Intel® Corporation. https://software.intel.com/en-us/vtune-help-cpu-metrics-reference

[Intel, 2023b] Intel (2023b). Intel® 64 and IA-32 Architectures Optimization Reference Manual. Intel® Corporation. https://software.intel.com/content/www/us/en/develop/download/intel-64-and-ia-32-architectures-optimization-reference-manual.html

[Jimenez & Lin, 2001] Jimenez, D. & Lin, C. (2001). Dynamic branch prediction with perceptrons. Proceedings HPCA Seventh International Symposium on High-Performance Computer Architecture, 197 – 206. https://doi.org/10.1109/HPCA.2001.903263

[Jin et al., 2012] Jin, G., Song, L., Shi, X., Scherpelz, J., & Lu, S. (2012). Understanding and detecting real-world performance bugs. Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '12, 77–88. https://doi.org/10.1145/2254064.2254075

[Kanev et al., 2015] Kanev, S., Darago, J. P., Hazelwood, K., Ranganathan, P., Moseley, T., Wei, G.-Y., & Brooks, D. (2015). Profiling a warehouse-scale computer. SIGARCH Comput. Archit. News, 43(3S), 158–169. https://doi.org/10.1145/2872887.2750392

[Kapoor, 2009] Kapoor, R. (2009). Avoiding the cost of branch misprediction. https://software.intel.com/en-us/articles/avoiding-the-cost-of-branch-misprediction

[Khuong & Morin, 2015] Khuong, P.-V. & Morin, P. (2015). Array layouts for comparison-based searching. https://arxiv.org/ftp/arxiv/papers/1509/1509.05053.pdf

[Leiserson et al., 2020] Leiserson, C. E., Thompson, N. C., Emer, J. S., Kuszmaul, B. C., Lampson, B. W., Sanchez, D., & Schardl, T. B. (2020). There's plenty of room at the top: What will drive computer performance after moore's law? Science, 368(6495). https://doi.org/10.1126/science.aam9744

[Liu et al., 2019] Liu, M., Sun, X., Varshney, M., & Xu, Y. (2019). Large-scale online experimentation with quantile metrics. https://arxiv.org/abs/1903.08762

[Luo et al., 2015] Luo, T., Wang, X., Hu, J., Luo, Y., & Wang, Z. (2015). Improving tlb performance by increasing hugepage ratio. 2015 15th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing, 1139 – 1142. https://doi.org/10.1109/CCGrid.2015.36

[Matteson & James, 2014] Matteson, D. S. & James, N. A. (2014). A nonparametric approach for multiple change point analysis of multivariate data. Journal of the American Statistical Association, 109(505), 334 – 345. https://doi.org/10.1080/01621459.2013.849605

[Mittal, 2016] Mittal, S. (2016). A survey of techniques for cache locking. ACM Transactions on Design Automation of Electronic Systems, 21. https://doi.org/10.1145/2858792

[Muła & Lemire, 2019] Muła, W. & Lemire, D. (2019). Base64 encoding and decoding at almost the speed of a memory copy. Software: Practice and Experience, 50(2), 89–97. https://doi.org/10.1002/spe.2777

[Mytkowicz et al., 2009] Mytkowicz, T., Diwan, A., Hauswirth, M., & Sweeney, P. F. (2009). Producing wrong data without doing anything obviously wrong! Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS XIV, 265–276. https://doi.org/10.1145/1508244.1508275

[Navarro-Torres et al., 2023] Navarro-Torres, A., Alastruey-Benedé, J., Ibáñez, P., & Viñals-Yúfera, V. (2023). Balancer: bandwidth allocation and cache partitioning for multicore processors. The Journal of Supercomputing, 79(9), 10252 – 10276. https://doi.org/10.1007/s11227-023-05070-0

[Navarro-Torres et al., 2019] Navarro-Torres, A. et al. (2019). Memory hierarchy characterization of SPEC CPU2006 and SPEC CPU2017 on the Intel Xeon Skylake-SP. PLOS ONE, 1 – 24. https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0220135

[Nowak & Bitzes, 2014] Nowak, A. & Bitzes, G. (2014). The overhead of profiling using pmu hardware counters. https://zenodo.org/record/10800/files/TheOverheadOfProfilingUsingPMUhardwareCounters.pdf

[Ottoni & Maher, 2017] Ottoni, G. & Maher, B. (2017). Optimizing function placement for large-scale data-center applications. Proceedings of the 2017 International Symposium on Code Generation and Optimization, CGO '17, 233–244. https://ieeexplore.ieee.org/document/7863743

[Panchenko et al., 2018] Panchenko, M., Auler, R., Nell, B., & Ottoni, G. (2018). BOLT: A practical binary optimizer for data centers and beyond. CoRR, abs/1807.06735. http://arxiv.org/abs/1807.06735

[Paoloni, 2010] Paoloni, G. (2010). How to Benchmark Code Execution Times on Intel® IA-32 and IA-64 Instruction Set Architectures. Intel® Corporation. https://www.intel.com/content/dam/www/public/us/en/documents/white-papers/ia-32-ia-64-benchmark-code-execution-paper.pdf

[Ren et al., 2010] Ren, G., Tune, E., Moseley, T., Shi, Y., Rus, S., & Hundt, R. (2010). Google-wide profiling: A continuous profiling infrastructure for data centers. IEEE Micro, 65 – 79. http://www.computer.org/portal/web/csdl/doi/10.1109/MM.2010.68

[Sasongko et al., 2023] Sasongko, M. A., Chabbi, M., Kelly, P. H. J., & Unat, D. (2023). Precise event sampling on amd versus intel: Quantitative and qualitative comparison. IEEE Transactions on Parallel and Distributed Systems, 34(5), 1594 – 1608. https://doi.org/10.1109/TPDS.2023.3257105

[Seznec & Michaud, 2006] Seznec, A. & Michaud, P. (2006). A case for (partially) tagged geometric history length branch prediction. J. Instr. Level Parallelism, 8. https://inria.hal.science/hal-03408381/document

[Sharma, 2016] Sharma, S. D. (2016). Hardware-assisted instruction profiling and latency detection. The Journal of Engineering, 2016, 367 – 376(9). https://digital-library.theiet.org/content/journals/10.1049/joe.2016.0127

[statista.com, 2018] statista.com (2018). Volume of data/information created worldwide from 2010 to 2025. Statista, Inc. https://www.statista.com/statistics/871513/worldwide-data-created/

[Suresh Srinivas, 2019] Suresh Srinivas, e. a. (2019). Runtime performance optimization blueprint: Intel® architecture optimization with large code pages. https://www.intel.com/content/www/us/en/develop/articles/runtime-performance-optimization-blueprint-intel-architecture-optimization-with-large-code.html

[Yasin, 2014] Yasin, A. (2014). A top-down method for performance analysis and counters architecture. 35 – 44. https://doi.org/10.1109/ISPASS.2014.6844459

附录 A. 减少测量噪声 (Reducing Measurement Noise)

以下是一些示例功能，它们可能导致性能测量的不确定性增加。有关完整讨论，请参见 Section 2.1。

动态频率缩放 (Dynamic Frequency Scaling)

动态频率缩放 (DFS): https://en.wikipedia.org/wiki/Dynamic_frequency_scaling[245] 是一种通过在运行要求苛刻任务时自动提高 CPU 运行频率来提升系统性能的技术。例如，英特尔 CPU 具有名为 睿频加速: https://en.wikipedia.org/wiki/Intel_Turbo_Boost[246] 的 DFS 实现功能，AMD CPU 则采用 Turbo Core: https://en.wikipedia.org/wiki/AMD_Turbo_Core[247] 功能。

以下示例展示了睿频加速对在 Intel® Core™ i5-8259U 上运行单线程工作负载的影响：

```
# 睿频加速启用
$ cat /sys/devices/system/cpu/intel_pstate/no_turbo
0
$ perf stat -e task-clock,cycles -- ./a.exe
    11984.691958  task-clock (msec) #    1.000 CPUs utilized
  32,427,294,227  cycles            #    2.706 GHz
      11.989164338 seconds time elapsed

# 睿频加速禁用
$ echo 1 | sudo tee /sys/devices/system/cpu/intel_pstate/no_turbo
1
$ perf stat -e task-clock,cycles -- ./a.exe
    13055.200832  task-clock (msec) #    0.993 CPUs utilized
  29,946,969,255  cycles            #    2.294 GHz
      13.142983989 seconds time elapsed
```

当睿频加速启用时，平均频率明显更高。

DFS 可以永久地在 BIOS 中禁用。[248] 要在 Linux 系统上以编程方式禁用 DFS 功能，您需要 root 权限。下面是如何实现这一点：

```
# 英特尔
echo 1 > /sys/devices/system/cpu/intel_pstate/no_turbo
# AMD
echo 0 > /sys/devices/system/cpu/cpufreq/boost
```

同时多线程 (Simultaneous Multithreading)

现代 CPU 内核通常采用 同时多线程 (SMT): https://en.wikipedia.org/wiki/Simultaneous_multithreading[249] 方式制造。这意味着在一个物理内核中，您可以同时执行两个线程。通常，体系结构状态: https://en.wikipedia.org/wiki/Archit

[245] 动态频率缩放 - https://en.wikipedia.org/wiki/Dynamic_frequency_scaling.
[246] 英特尔睿频加速 - https://en.wikipedia.org/wiki/Intel_Turbo_Boost.
[247] AMD Turbo Core - https://en.wikipedia.org/wiki/AMD_Turbo_Core.
[248] 英特尔睿频加速常见问题解答 - https://www.intel.com/content/www/us/en/support/articles/000007359/processors/intel-core-processors.html.
[249] SMT - https://en.wikipedia.org/wiki/Simultaneous_multithreading.

ectural_state[250] 会被复制，但执行资源（ALU、缓存等）不会。这意味着如果我们在同一个内核上以“同时”方式运行两个独立的进程（在不同的线程中），它们会相互争夺资源，例如缓存空间。

SMT 可以永久地在 BIOS 中禁用。[251] 要在 Linux 系统上以编程方式禁用 SMT，您需要 root 权限。下面是如何在每个内核中关闭一个兄弟线程：

```
echo 0 > /sys/devices/system/cpu/cpuX/online
```

可以在以下文件中找到 CPU 线程的兄弟对：

```
/sys/devices/system/cpu/cpuN/topology/thread_siblings_list
```

例如，在具有 4 个内核和 8 个线程的 Intel® Core™ i5-8259U 上：

```
# 所有 8 个硬件线程均已启用：
$ lscpu
...
CPU(s):              8
On-line CPU(s) list: 0-7
...
$ cat /sys/devices/system/cpu/cpu0/topology/thread_siblings_list
0,4
$ cat /sys/devices/system/cpu/cpu1/topology/thread_siblings_list
1,5
$ cat /sys/devices/system/cpu/cpu2/topology/thread_siblings_list
2,6
$ cat /sys/devices/system/cpu/cpu3/topology/thread_siblings_list
3,7

# 在内核 0 上禁用 SMT
$ echo 0 | sudo tee /sys/devices/system/cpu/cpu4/online
0
$ lscpu
CPU(s):               8
On-line CPU(s) list:  0-3,5-7
Off-line CPU(s) list: 4
...
$ cat /sys/devices/system/cpu/cpu0/topology/thread_siblings_list
0
```

缩放调速器 (Scaling Governor)

Linux 内核可以控制 CPU 频率用于不同的目的。其中一个目的是节能，在这种情况下，Linux 内核内部的调速器[252]可以决定降低 CPU 运行频率。对于性能测量，建议将调速器策略设置为“性能”，以避免低于标称时钟频率。下面是如何将其设置为所有内核：

[250] 体系结构状态 - https://en.wikipedia.org/wiki/Architectural_state.
[251] “如何禁用超线程” - https://www.pcmag.com/article/314585/how-to-disable-hyperthreading.
[252] Linux CPU 频率调节器文档：https://www.kernel.org/doc/Documentation/cpu-freq/governors.txt.

```
for i in /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor
do
  echo performance > $i
done
```

CPU 亲和性 (CPU Affinity)

处理器亲和性: https://en.wikipedia.org/wiki/Processor_affinity[253] 允许将进程绑定到特定 CPU 内核。在 Linux 中，可以使用 `taskset`: https://linux.die.net/man/1/taskset[254] 工具实现这一点。这里

```
# 无亲和性
$ perf stat -e context-switches,cpu-migrations -r 10 -- a.exe
               151      context-switches
                10      cpu-migrations

# 进程绑定到 CPU0
$ perf stat -e context-switches,cpu-migrations -r 10 -- taskset -c 0 a.exe
               102      context-switches
                 0      cpu-migrations
```

请注意，`cpu-migrations` 的数量变为 `0`，即进程永远不会离开 `core0`。

或者，您可以使用 cset: https://github.com/lpechacek/cpuset[255] 工具仅为要基准测试的程序预留 CPU。如果使用 `Linux perf`，请至少保留两个内核，以便 `perf` 在一个内核上运行，您的程序在另一个内核上运行。以下命令将从 N1 和 N2 中移动所有线程（`-k on` 表示即使内核线程也会被移动出去）：

```
$ cset shield -c N1,N2 -k on
```

以下命令将在隔离的 CPU 上运行 `--` 后面的命令：

```
$ cset shield --exec -- perf stat -r 10 <cmd>
```

进程优先级 (Process Priority)

在 Linux 中，可以使用 `nice` 工具提高进程优先级。通过提高优先级，进程可以获得更多 CPU 时间，并且 Linux 调度器会比具有正常优先级的进程更青睐它。优先级范围从 `-20`（最高优先级值）到 `19`（最低优先级值），默认值为 `0`。

请注意在上一个示例中，基准测试进程的执行被操作系统中断超过 100 次。如果我们通过使用 `sudo nice -n -N` 运行基准测试来提高进程优先级：

```
$ perf stat -r 10 -- sudo nice -n -5 taskset -c 1 a.exe
    0   context-switches
    0   cpu-migrations
```

请注意，上下文切换的数量变为 `0`，因此该进程不间断地接收所有计算时间。

[253] 处理器关联性 - https://en.wikipedia.org/wiki/Processor_affinity.
[254] `taskset` 手册 - https://linux.die.net/man/1/taskset.
[255] `cpuset` 手册 - https://github.com/lpechacek/cpuset.

文件系统缓存 (Filesystem Cache)

通常，会分配一部分主内存来缓存文件系统内容，包括各种数据。这减少了应用程序访问磁盘的次数。以下示例说明了文件系统缓存如何影响简单 `git status` 命令的运行时间：

```
# 清空文件系统缓存
$ echo 3 | sudo tee /proc/sys/vm/drop_caches && sync && time -p git status
real 2,57
# 预热文件系统缓存
$ time -p git status
real 0,40
```

可以通过运行以下两个命令清除当前的文件系统缓存：

```
$ echo 3 | sudo tee /proc/sys/vm/drop_caches
$ sync
```

或者，您可以进行一次干运行以预热文件系统缓存，并将其排除在测量之外。此干运行可以与基准测试输出的验证结合使用。

附录 B. LLVM 向量化 (Vectorizer)

本章节描述了截至 2020 年 Clang 编译器内部 LLVM 循环向量化的状态。内部循环向量化是将最内层循环中的代码转换为使用跨多个循环迭代的向量的代码的过程。SIMD 向量中的每个通道对连续的循环迭代执行独立算术运算。通常，循环不会处于干净状态，向量化必须猜测和假设缺少的信息并会在运行时检查细节。如果假设错误，向量化会退回到运行标量循环。下面的示例突出了一些 LLVM 向量化支持的代码模式。

循环次数未知

LLVM 循环向量化支持循环次数未知的情况。在下面的循环中，迭代开始和结束点未知，向量化有一个机制可以向量化不以零开始的循环。在这个例子中，n 可能不是向量宽度的倍数，向量化必须将最后几个迭代作为标量代码执行。保留循环的标量副本会增加代码大小。

```
void bar(float* A, float* B, float K, int start, int end) {
  for (int i = start; i < end; ++i)
    A[i] *= B[i] + K;
}
```

指针的运行时检查

在下面的示例中，如果指针 A 和 B 指向连续的地址，则向量化代码是非法的，因为某些 A 的元素会在从数组 B 读取之前写入。

一些程序员使用 restrict 关键字通知编译器指针是不相交的，但在我们的例子中，LLVM 循环向量化无法知道指针 A 和 B 是唯一的。循环向量化通过放置代码来处理这个循环，该代码在运行时检查数组 A 和 B 是否指向不相交的内存位置。如果数组 A 和 B 重叠，则执行循环的标量版本。

```
void bar(float* A, float* B, float K, int n) {
  for (int i = 0; i < n; ++i)
    A[i] *= B[i] + K;
}
```

归约

在这个例子中，求和变量被循环的连续迭代使用。通常，这会阻止向量化，但向量化可以检测到 sum 是一个归约变量。变量 sum 成为一个整数向量，在循环结束时，数组的元素被相加以创建正确的结果。LLVM 循环向量化支持许多不同的归约操作，例如加、乘、异或、与和或。

```
int foo(int *A, int n) {
  unsigned sum = 0;
  for (int i = 0; i < n; ++i)
    sum += A[i] + 5;
return sum;
}
```

使用 -ffast-math 时，LLVM 循环向量化支持浮点归约操作。

归纳

在这个例子中，归纳变量 i 的值被保存到一个数组中。LLVM 循环向量化知道如何向量化归纳变量。

```
void bar(float* A, int n) {
  for (int i = 0; i < n; ++i)
    A[i] = i;
}
```

If 转换

LLVM 循环向量器能够“平铺”代码中的 IF 语句并生成单个指令流。向量化支持最内层循环中的任何控制流。最内层循环可能包含复杂的 IF、ELSE 甚至 GOTO 的嵌套。

```
int foo(int *A, int *B, int n) {
  unsigned sum = 0;
  for (int i = 0; i < n; ++i)
    if (A[i] > B[i])
      sum += A[i] + 5;
  return sum;
}
```

指针归纳变量 {.unnumbered .unlisted}

此示例使用标准 C++ 库中的 `std::accumulate` 函数。此循环使用 C++ 迭代器，它们是指针，而不是整数索引。LLVM 循环向量化器检测指针归纳变量并可以向量化此循环。此功能很重要，因为许多 C++ 程序使用迭代器。

```
int baz(int *A, int n) {
  return std::accumulate(A, A + n, 0);
}
```

反向迭代器

LLVM 循环向量化器可以向量化反向计数的循环。

```
int foo(int *A, int n) {
  for (int i = n; i > 0; --i)
    A[i] +=1;
}
```

Scatter / Gather

LLVM 循环向量化器可以向量化成为分散/收集内存的标量指令序列的代码。

```
int foo(int * A, int * B, int n) {
  for (intptr_t i = 0; i < n; ++i)
    A[i] += B[i * 4];
}
```

在许多情况下，成本模型会认为这种转换是无利可图的。

混合类型的向量化

LLVM 循环向量化器可以向量化具有混合类型的程序。向量化器成本模型可以估计类型转换的成本并决定向量化是否有利可图。

```
int foo(int *A, char *B, int n) {
  for (int i = 0; i < n; ++i)
    A[i] += 4 * B[i];
}
```

函数调用的向量化

LLVM 循环向量化器可以向量化内在数学函数。有关这些功能的列表，请参见下表。

```
pow         exp         exp2
sin         cos         sqrt
log         log2        log10
fabs        floor       ceil
fma         trunc       nearbyint
fmuladd
```

向量化过程中的部分展开

现代处理器具有多个执行单元，只有包含高度并行性的程序才能充分利用机器的全部宽度。LLVM 循环向量化器通过执行循环的部分展开来增加指令级并行度（ILP）。

在下面的示例中，整个数组累积到变量 sum 中。这是低效的，因为处理器只能使用单个执行端口。通过展开代码，循环向量化器允许同时使用两个或多个执行端口。

```
int foo(int *A, int n) {
  unsigned sum = 0;
  for (int i = 0; i < n; ++i)
    sum += A[i];
  return sum;
}
```

LLVM 循环向量化器使用成本模型来决定何时展开循环是有利的。展开循环的决定取决于寄存器压力和生成的代码大小。

SLP 向量化

SLP（超字级并行 Superword-Level Parallelism）向量化器试图将多个标量操作粘合在一起形成向量操作。它自下而上地处理代码，跨越基本块，以寻找要组合的标量。SLP 向量化的目标是将类似的独立指令组合成向量指令。内存访问、算术运算、比较运算都可以使用这种技术进行向量化。例如，以下函数对其输入 (a1, b1) 和 (a2, b2) 执行非常相似的操作。基本块向量化器可以将以下函数组合成向量运算。

```
void foo(int a1, int a2, int b1, int b2, int *A) {
  A[0] = a1*(a1 + b1);
  A[1] = a2*(a2 + b2);
  A[2] = a1*(a1 + b1);
```

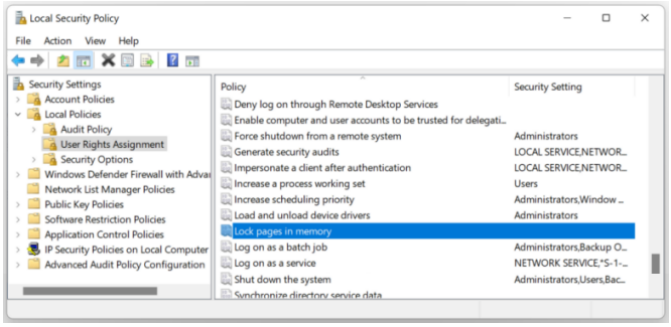
```
  A[3] = a2*(a2 + b2);
}
```

附录 C. 启用大页面

14.1 Windows

要在 Windows 上使用大页面，需要启用 SeLockMemoryPrivilege 安全策略 (链接到微软文档: https://docs.microsoft.com/en-us/windows/security/threat-protection/security-policy-settings/lock-pages-in-memory). 这可以通过 Windows API 以编程方式完成，也可以通过安全策略 GUI 完成。

1. 点击开始 -> 搜索 “secpol.msc”，启动它。
2. 在左侧选择 “Local Policies” -> “User Rights Assignment” (“本地策略” -> “用户权利分配”)，然后双击 “锁定内存中的页面”。



3. 添加您的用户并重新启动机器。
4. 使用 RAMMap: https://docs.microsoft.com/en-us/sysinternals/downloads/rammap 工具检查运行时是否使用了大页面。

在代码中使用大页面：

```
void* p = VirtualAlloc(NULL, size, MEM_RESERVE |
                                   MEM_COMMIT |
                                   MEM_LARGE_PAGES,
                       PAGE_READWRITE);
...
VirtualFree(ptr, 0, MEM_RELEASE);
```

14.2 Linux

在 Linux 操作系统上，应用程序可以使用大页面的两种方式：显式和大页面透明分配。

显式大页面

显式大页面可以在启动时或运行时预留。要在启动时强制 Linux 内核分配 128 个大页面，请运行以下命令：

```
$ echo "vm.nr_hugepages = 128" >> /etc/sysctl.conf
```

要显式分配固定数量的大页面，可以使用 libhugetlbfs: https://github.com/libhugetlbfs/libhugetlbfs。以下命令预分配 128 个大页面。

```
$ sudo apt install libhugetlbfs-bin
```

```
$ sudo hugeadm --create-global-mounts
$ sudo hugeadm --pool-pages-min 2M:128
```

这大致相当于执行以下命令，不需要 libhugetlbfs（参见内核文档：链接到内核文档: https://www.kernel.org/doc/Documentation/vm/hugetlbpage.txt):

```
$ echo 128 > /proc/sys/vm/nr_hugepages
$ mount -t hugetlbfs                                                    \
    -o uid=<value>,gid=<value>,mode=<value>,pagesize=<value>,size=<value>,\
    min_size=<value>,nr_inodes=<value> none /mnt/huge
```

您应该可以在 /proc/meminfo 中观察到效果。请注意，这是一个系统范围的视图，而不是针对每个进程:

```
$ watch -n1 "cat /proc/meminfo  | grep huge -i"
AnonHugePages:      2048 kB
ShmemHugePages:        0 kB
FileHugePages:         0 kB
HugePages_Total:     128    <== 128 huge pages allocated
HugePages_Free:      128
HugePages_Rsvd:        0
HugePages_Surp:        0
Hugepagesize:       2048 kB
Hugetlb:          262144 kB <== 256MB of space occupied
```

开发人员可以在代码中通过调用带有 MAP_HUGETLB 标志的 mmap 来利用显式大页面（完整示例[256]）:

```
void ptr = mmap(nullptr, size, PROT_READ | PROT_WRITE,
                MAP_PRIVATE | MAP_ANONYMOUS | MAP_HUGETLB, -1, 0);
...
munmap(ptr, size);
```

其他替代方案包括:

- 使用来自已挂载的 hugetlbfs 文件系统的文件进行 mmap (示例代码[257])。
- 使用 SHM_HUGETLB 标志的 shmget (示例代码[258]).

透明大页内存

要在 Linux 上允许应用程序使用透明大页 (THP)，需要确保 /sys/kernel/mm/transparent_hugepage/enabled 设置为 always 或 madvise。前者启用系统范围的 THP 使用，而后者则将对哪些内存区域应使用 THP 的控制权交给用户代码，从而避免消耗更多内存资源的风险。以下是使用 madvise 方法的示例:

```
void ptr = mmap(nullptr, size, PROT_READ | PROT_WRITE | PROT_EXEC,
                MAP_PRIVATE | MAP_ANONYMOUS, -1 , 0);
madvise(ptr, size, MADV_HUGEPAGE);
...
munmap(ptr, size);
```

[256] MAP_HUGETLB 示例 - https://github.com/torvalds/linux/blob/master/tools/testing/selftests/mm/map_hugetlb.c.
[257] 已挂载的 hugetlbfs 文件系统 - https://github.com/torvalds/linux/blob/master/tools/testing/selftests/mm/hugepage-mmap.c.
[258] SHM_HUGETLB 示例 - https://github.com/torvalds/linux/blob/master/tools/testing/selftests/vm/hugepage-shm.c.

您可以在 /proc/meminfo 下的 AnonHugePages 中观察到系统范围的效果：

```
$ watch -n1 "cat /proc/meminfo  | grep huge -i"
AnonHugePages:     61440 kB     <== 30 transparent huge pages are in use
HugePages_Total:     128
HugePages_Free:      128        <== explicit huge pages are not used
```

此外，开发人员可以通过查看针对其进程的特定 smaps 文件来观察其应用程序如何利用 EHP 和/或 THP：

```
$ watch -n1 "cat /proc/<PID_OF_PROCESS>/smaps"
```

附录 D. Intel 处理器跟踪

Intel 处理器跟踪（PT）是一种 CPU 功能，通过将数据包编码为高度压缩的二进制格式记录程序执行，可以在每条指令上附带时间戳，用于重构执行流。PT 具有广泛的覆盖范围和相对较小的开销，[259] 通常低于 5%。其主要用途是事后分析和排查性能故障的根本原因。

工作流程

与采样技术类似，PT 不需要对源代码进行任何修改。收集跟踪的全部需要就是在支持 PT 的工具下运行程序。一旦启用了 PT 并启动了基准测试，分析工具就会开始将跟踪数据包写入 DRAM。

与 LBR（Last Branch Records）类似，Intel PT 通过记录分支来工作。在运行时，每当 CPU 遇到任何分支指令时，PT 就会记录该分支的结果。对于简单的条件跳转指令，CPU 会记录其是否被执行（T）或未被执行（NT），仅使用 1 位。对于间接调用，PT 将记录目标地址。请注意，由于我们静态知道无条件分支的目标，因此会忽略无条件分支。

示例中展示了一小段指令序列的编码，如图 86 所示。诸如 PUSH、MOV、ADD 和 CMP 等指令被忽略，因为它们不会改变控制流。但是，JE 指令可能会跳转到 .label，因此需要记录其结果。稍后存在一个间接调用，需要保存目标地址。

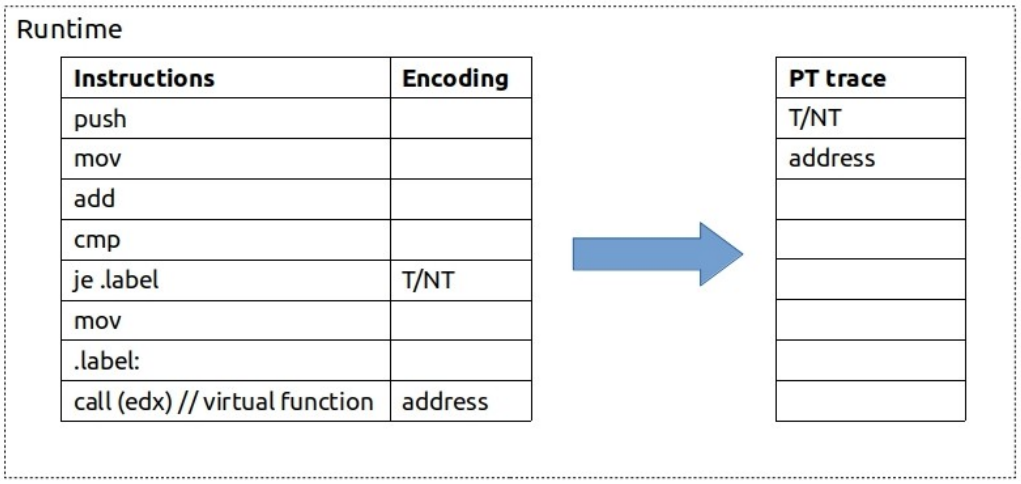


Figure 86: Intel 处理器跟踪编码

在分析时，我们需要将应用程序二进制文件和收集到的 PT 跟踪数据合并在一起。一个软件解码器需要应用程序二进制文件来重构程序的执行流程。它从入口点开始，然后使用收集到的跟踪数据作为查找参考来确定控制流。图 87 展示了 Intel 处理器跟踪的解码示例。假设 PUSH 指令是应用程序二进制文件的入口点。然后 PUSH、MOV、ADD 和 CMP 等指令被按原样重构，而无需查看编码的跟踪数据。稍后，软件解码器遇到一个 JE 指令，这是一个条件分支，需要查找结果。根据图 87 中的跟踪数据，JE 被执行（T），因此我们跳过下一个 MOV 指令并转到 CALL 指令。同样，CALL(edx) 是一个改变控制流的指令，因此我们在编码的跟踪数据中查找目标地址，即 0x407e1d8。在我们的程序运行时执行的指令用黄色突出显示。请注意，这是程序执行的 精确 重构；我们没有跳过任何指令。稍后，我们可以使用调试信息将汇编指令映射回源代码，并记录逐行执行的源代码日志。

[259] 有关英特尔 PT 额外开销的更多信息，请参见 [Sharma, 2016]。

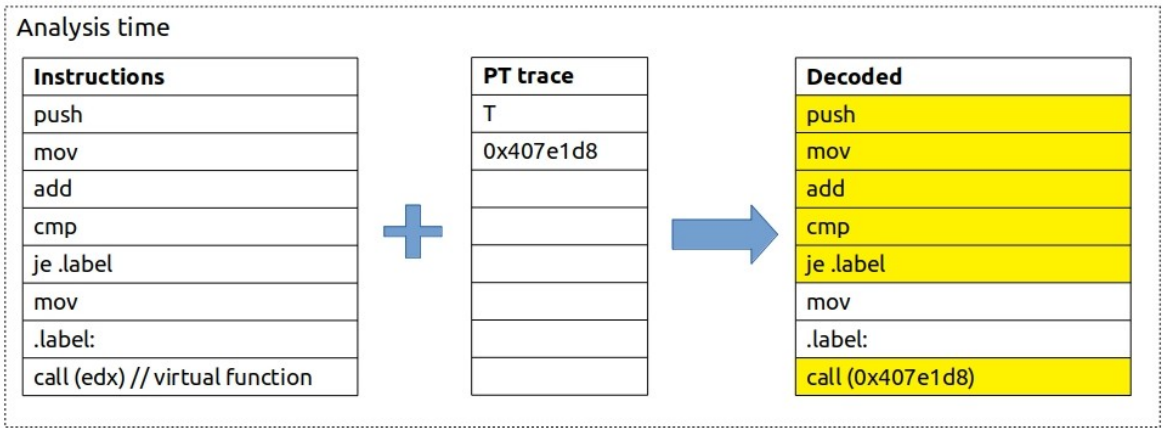


Figure 87: Intel 处理器跟踪解码

时间数据包

使用 Intel PT，不仅可以跟踪执行流，还可以跟踪时间信息。除了保存跳转目标外，PT 还可以发出时间数据包。图 88 提供了时间数据包如何用于恢复指令时间戳的可视化示例。与前面的示例类似，我们首先看到 JNZ 没有被执行，因此我们将其及其上方的所有指令的时间戳更新为 0 纳秒。然后我们看到一个 2 纳秒的时间更新和 JE 被执行，因此我们将其及其上方的所有指令（以及下方的 JNZ）的时间戳更新为 2 纳秒。之后是一个间接调用，但没有附加时间数据包，因此我们不更新时间戳。然后我们看到经过了 100 纳秒，并且 JB 没有被执行，因此我们将其上方的所有指令的时间戳更新为 102 纳秒。

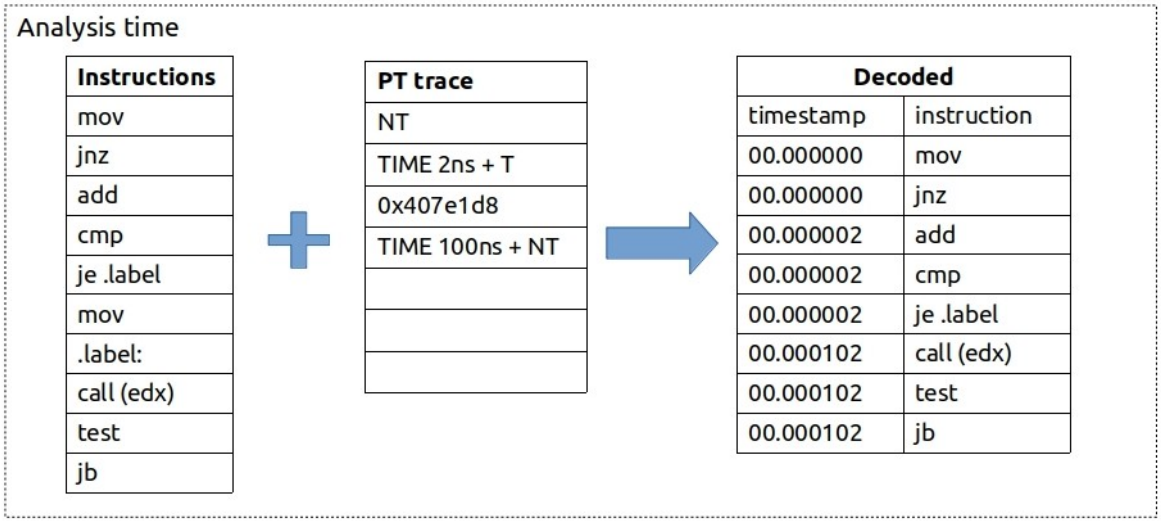


Figure 88: Intel 处理器跟踪时间数据包

在图 88 中展示的示例中，指令数据（控制流）完全准确，但时间信息不太准确。显然，CALL(edx)、TEST 和 JB 指令并不是同时发生的，但我们没有更准确的时间信息。具有时间戳使我们能够将程序的时间间隔与系统中的另一个事件对齐，并且很容易与挂钟时间进行比较。在某些实现中，跟踪时间可以通过一个循环精确模式进一步改进，其中硬件记录了常规数据包之间的周期计数（有关更多详细信息，请参阅 [Intel, 2023b, 第 3C 卷，第 36 章]）.

收集和解码跟踪

使用 Linux 的 `perf` 工具可以轻松地收集 Intel PT 跟踪数据：

```
$ perf record -e intel_pt/cyc=1/u ./a.out
```

在上面的命令行中，我们要求 PT 机制每个周期更新一次时间信息。但是很可能，这不会显著增加我们的准确性，因为只有在与另一个控制流数据包配对时，才会发送时间数据包。

收集完跟踪数据后，可以通过执行以下命令来获取原始的 PT 跟踪数据：

```
$ perf report -D > trace.dump
```

PT 在发出时间数据包之前会捆绑最多 6 个条件分支。自 Intel Skylake CPU 一代以来，时间数据包包含自上一个数据包以来经过的周期数。然后，如果我们查看 `trace.dump`，可能会看到类似以下的内容：

```
000073b3: 2d 98 8c  TIP 0x8c98     // 目标地址（IP）
000073b6: 13        CYC 0x2        // 时间更新
000073b7: c0        TNT TNNNNN (6) // 6 个条件分支
000073b8: 43        CYC 0x8        // 经过 8 个周期
000073b9: b6        TNT NTTNTT (6)
```

以上是显示的原始 PT 数据包，对性能分析并不是非常有用。要将处理器跟踪数据解码为人类可读形式，可以执行以下命令：

```
$ perf script --ns --itrace=i1t -F time,srcline,insn,srccode
```

以下是可能获得的解码跟踪的示例：

```
时间戳          源代码行    指令                源代码
...
253.555413143:  a.cpp:24  call 0x35c       foo(arr, j);
253.555413143:  b.cpp:7   test esi, esi    for (int i = 0; i <= n; i++)
253.555413508:  b.cpp:7   js 0x1e
253.555413508:  b.cpp:7   movsxd rsi, esi
...
```

以上只是来自长时间执行日志的一个小片段。在此日志中，我们有跟踪 每个 执行的指令，而我们的程序正在运行时。我们可以真正观察程序所采取的每一步。这是进一步功能和性能分析的非常强大的基础。

使用情况

1. **分析性能故障**：由于 PT 捕获了整个指令流，因此可以分析应用程序未响应的小时间段内发生了什么。在 easyperf 博客的一篇文章[260]中可以找到更详细的示例。
2. **事后调试**：PT 跟踪数据可以由像 gdb 这样的传统调试器进行回放。除此之外，PT 还提供了调用堆栈信息，即使堆栈已损坏，该信息也始终有效。[261]可以在远程机器上收集 PT 跟踪数据，然后在离线状态下进行分析。这在问题难以重现或系统访问受限时特别有用。
3. **审查程序的执行**：
    - 我们可以立即知道是否未执行某个代码路径。

[260] 使用英特尔 PT 分析性能故障 - https://easyperf.net/blog/2019/09/06/Intel-PT-part3。
[261] 使用英特尔 PT 进行事后调试 - https://easyperf.net/blog/2019/08/30/Intel-PT-part2。

- 多亏了时间戳，可以计算在尝试获取锁时等待的时间，等等。
- 通过检测特定指令模式进行安全缓解。

磁盘空间和解码时间

即使考虑到跟踪的压缩格式，编码后的数据也会占用大量磁盘空间。通常，每个指令少于 1 个字节，但考虑到 CPU 执行指令的速度，它仍然很多。取决于工作负载，CPU 以 100 MB/s 的速度编码 PT 非常常见。解码的跟踪可能很容易增加十倍（~1GB/s）。这使得 PT 不适用于长时间运行的工作负载。但即使是在大工作负载上，运行一小段时间也是负担得起的。在这种情况下，用户只能在故障发生期间附加到正在运行的进程。或者他们可以使用循环缓冲区，新跟踪将覆盖旧跟踪，即始终拥有最近 10 秒左右的跟踪。

用户可以通过多种方式进一步限制收集。他们可以限制仅在用户/内核空间代码上收集跟踪。此外，还有一个地址范围过滤器，因此可以动态地选择加入和退出跟踪以限制内存带宽。这使我们能够跟踪单个函数甚至单个循环。

解码 PT 跟踪可能需要很长时间。在 Intel Core i5-8259U 机器上，对于运行 7 毫秒的工作负载，编码的 PT 跟踪大约消耗 1MB 的磁盘空间。使用 `perf script` 解码此跟踪需要大约 20 秒。使用 `perf script -F time,ip,sym,symoff,insn` 的解码输出大约占用 1.3GB 的磁盘空间。截至 2020 年 2 月，使用 `perf script -F` 以及 `+srcline` 或 `+srccode` 解码跟踪变得非常慢，不适合日常使用。应该改进 Linux perf 的实现。

英特尔 PT 参考资料和链接

- 英特尔 ® 64 和 IA-32 架构软件开发人员手册 [Intel, 2023b, 第 3 卷 C，第 36 章]。
- 白皮书“硬件辅助指令分析和延迟检测”[Sharma, 2016]。
- Andi Kleen 在 LWN 上的文章，网址：https://lwn.net/Articles/648154。
- 英特尔 PT 微型教程，网址：https://sites.google.com/site/intelptmicrotutorial/。
- simple_pt: Linux 上的简单英特尔 CPU 处理器跟踪，网址： https://github.com/andikleen/simple-pt。
- Linux 内核中的英特尔 PT 文档，网址： https://github.com/torvalds/linux/blob/master/tools/perf/Documentation/intel-pt.txt。
- 英特尔处理器跟踪备忘单，网址：http://halobates.de/blog/p/410。